Q1: Multi-Task CNN on Fashion-MNIST (75 Marks)

Objective

Convolutional Neural Networks (CNNs) are widely used for image-based tasks such as classification, regression, and object detection. In this assignment, you will implement a single **multi-task CNN** that jointly performs:

(a) **Classification** of Fashion-MNIST images into 10 clothing categories, and

(b) **Regression** of a continuous "ink" value corresponding to the normalized pixel intensity of each image.

The model will be trained using a joint loss:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{classification}} + \lambda_2 \mathcal{L}_{\text{regression}},$$

where $\mathcal{L}_{\text{classification}}$ is Cross-Entropy loss and $\mathcal{L}_{\text{regression}}$ is Mean Squared Error (MSE). You will explore the effect of λ_1 and λ_2 on the trade-off between classification and regression performance.

All experiments must be logged using Weights & Biases (wandb).

Dataset Description (Fashion-MNIST)

The Fashion-MNIST dataset (Zalando Research) contains 70,000 grayscale images (28×28) divided into 10 clothing classes:

```
0: T-shirt/top, 1: Trouser, 2: Pullover, 3: Dress, 4: Coat, 5: Sandal,
            6: Shirt, 7: Sneaker, 8: Bag, 9: Ankle boot.
```

Use the training split for model training and validation, and the provided test split for final evaluation.

Data Loading and Preprocessing (15 Marks)

1. **Dataset loading:** Implement a function `load_fashion_data()` that:
   - Loads the Fashion-MNIST dataset from Kaggle or torchvision,
   - Splits the original training set into `train` and `val` (e.g., 90/10 ratio),
   - Ensures no overlap or data leakage between splits.
2. **Custom Dataset class:** Implement `FashionMNISTDataset` that returns tuples `(image, class_label, ink_target)`:
   - **image:** normalized tensor using the mean and std of Fashion-MNIST,
   - **class_label:** the integer label $y \in \{0, \ldots, 9\}$,
   - **ink_target:** the normalized pixel intensity (average pixel value of the image), representing how much "ink" is used.

3. **Augmentations:** Use **light augmentations** (e.g., random crop, small rotation) for the training set only to encourage generalization. Keep validation and test sets unaltered.

Model Implementation: Multi-Task CNN (25 Marks)

1. **Architecture (7.5 Marks):** Implement a shared convolutional backbone with 2–3 Conv–BatchNorm–ReLU–Pool blocks and dropout. The model should then branch into two separate heads:
   - **Classification head:** outputs logits $[B, 10]$,
   - **Regression head:** outputs scalar values $[B, 1]$.
2. **Forward pass (7.5 Marks):** The `forward()` method should:
   - Pass inputs through shared convolutional layers,
   - Output both classification logits and regression predictions,
   - Optionally return intermediate feature maps for visualization.
3. **Joint loss (10 Marks):** Implement the total loss as:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{CE}} + \lambda_2 \mathcal{L}_{\text{MSE}}.$$

   Experiment with different values of λ_1 and λ_2 (e.g., 1:1, 2:1, 1:2, 0.5:1, etc.) to observe how they influence the model's performance on both tasks.

Hyperparameter Tuning and wandb Logging (20 Marks)

1. **Hyperparameters (7.5 Marks):** Explore key hyperparameters:
   - learning rate, dropout rate, optimizer choice (SGD, Adam, AdamW),
   - number of convolutional layers/filters, batch size, and
   - weighting factors λ_1 and λ_2.
2. **wandb Logging (Mandatory):** Each run must be logged on **Weights & Biases (wandb)**. Record:
   - Configuration (hyperparameters, λ_1, λ_2),
   - Training/validation loss curves (total, CE, MSE),
   - Validation metrics (accuracy, MAE, RMSE),
   - Final test performance and visualizations.
3. **Training runs (7.5 Marks):** Conduct at least **five distinct runs**, varying hyperparameters and λ_1, λ_2. Plot the training and validation losses for all runs (preferably via wandb line charts).

4. **Model selection (5 Marks):** Identify:

   - The model with the highest validation classification accuracy,
   - The model with the lowest validation regression RMSE.

   Report the corresponding test metrics and discuss how varying λ_1, λ_2 affected the trade-off between the two objectives.

Feature Map Visualization (15 Marks)

1. **Intermediate feature maps (7.5 Marks):** Modify `forward()` (or use hooks) to extract intermediate outputs after each convolutional block. Visualize feature maps from each layer for any three test images.

2. **Interpretation (7.5 Marks):** Describe the kinds of features captured at different layers (edges, textures, shapes, clothing regions) and how they help both classification and ink regression.

Deliverables

- **wandb Dashboard:** include a shareable project link in your report.
- **Report (PDF):** Describe your approach, hyperparameters, wandb plots, best models, and feature map visualizations. Discuss the observed trade-off between classification and regression performance as λ_1, λ_2 vary.
- **Metrics:**
  - Classification — Accuracy (and optionally F1-score)
  - Regression — MAE and RMSE